ADR_008 庫存熱點：原子條件更新與演進路線

> 狀態：已接受
> 日期：2026-10-14

背景

搶購時大量請求同時修改同一筆搶購庫存（非功能需求：1 萬人搶 500 件、零超賣、p99 低於 1 秒）。資料庫修改一筆資料時會鎖住它，這筆資料每秒能處理的修改次數，就是搶購的上限。在 ADR_005 的單一交易下，鎖要持有到整個結帳交易提交，問題更明顯。

搶購庫存與一般庫存是不同的資料（需求 3.3），搶購的熱點不會影響原價購買。

考慮過的選項

在資料庫上處理

選項	在熱點下的表現
悲觀鎖：先鎖定、讀出、檢查、再更新	一個接一個處理，鎖持有時間最長
樂觀鎖：以版本號檢查，衝突則重試	低競爭時很好；高競爭時大量衝突與重試，反而更差
原子條件更新：一條指令完成「足夠才扣減」，以影響筆數判斷成敗	不需要先讀再寫，是資料庫層最好的做法

減少打到資料庫的請求：售完快速拒絕、先檢查限購、限流。

把熱點移出資料庫：Redis 預扣、排隊削峰、庫存分桶。

決定

第一版以原子條件更新扣減庫存，並放在結帳交易的最後一步，縮短鎖的持有時間。

搶購階段依序演進，每一步先壓測，達到目標即停止：

1. 售完快速拒絕、先檢查限購、限流
2. 改用 Redis 預扣，結帳同時改為補償方式（觸發 ADR_005 的檢討條件）
3. 若尖峰仍壓垮後續流程，加入排隊削峰

另做對照實驗：在相同情境下比較悲觀鎖、樂觀鎖、原子條件更新的吞吐量、回應時間與失敗率，結果另寫一份 ADR。

不採用庫存分桶：它解決的問題 Redis 預扣能解決得更好，又讓售完的判斷變複雜。

後果

- 好處：每一步的改動都有量測數據支持；最終架構的每個元件都知道為什麼存在
- 代價：搶購相關的程式會經歷數次改寫
- 重新檢討的條件：每完成一個演進步驟就以壓測結果檢視，並記錄為新的 ADR